

```

// Vulkan 1.4 피처 활성화 설정 예시
VkPhysicalDeviceShaderFloat16Int8Features float16Features{};
float16Features.sType =
VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_SHADER_FLOAT16_INT8_FEATURES;
float16Features.shaderFloat16 = VK_TRUE;

VkPhysicalDeviceFeatures2 deviceFeatures2{};
deviceFeatures2.sType = VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FEATURES_2;
deviceFeatures2.pNext = &float16Features;

#version 460
#extension GL_EXT_shader_16bit_storage : require
#extension GL_KHR_shader_subgroup_arithmetic : require

layout(local_size_x = 256, local_size_y = 1, local_size_z = 1) in;

// 패킹된 데이터 버퍼 (FP32 자리에 FP16x2가 중첩되어 들어감)
layout(set = 0, binding = 0) readonly buffer InputBuffer {
    uint packedData[]; // 32비트 공간에 16비트 값 2개씩 스토리지 패킹
};

layout(set = 0, binding = 1) writeonly buffer OutputBuffer {
    float resultData[]; // 최종 IEEE 754 FP32 규격 연산 결과
};

void main() {
    uint idx = gl_GlobalInvocationID.x;

    // 1. 32비트 레지스터에서 FP16 두 개를 언팩 (Packed FP16 -> 두 개의 하프 프리시전 값)
    uint raw32 = packedData[idx];
    vec2 unpacked_f16 = unpackHalf2x16(raw32);

    // 2. RDNA FP32/FP16 유닛에서 동시 연산 수행 (SIMD 웨이브 연산 상정)
    // 예: 두 패킹된 데이터 간의 곱셈 및 스케일링
    vec2 computed_f16 = unpacked_f16 * unpacked_f16;

    // 3. IEEE 754 FP32 규격 변환 및 정밀도(Mantissa/Exponent) 매핑 정합성 유지
    // FP16 값을 표준 FP32 스칼라로 확장 승격하면서 IEEE 754 바이트 규약 정렬
    float final_f32 = float(computed_f16.x) + float(computed_f16.y);

    // 4. 결과 스토어
    resultData[idx] = final_f32;
}

```